

NetSentinel

Complete Knowledge Guide

Everything You Need to Know for Your Project Defense

Table of Contents

1. Networking Fundamentals (OSI, TCP/IP, Protocols)
2. Network Security Concepts (IDS, IPS, Attacks)
3. The UNSW-NB15 Dataset (In Depth)
4. Machine Learning Models Used
5. Explainable AI (SHAP, LIME, XAI)
6. Tools & Libraries (Scapy, Streamlit, etc.)
7. NetSentinel Architecture & Design
8. Research Context & Related Work
9. Defense Q&A; — 50+ Questions & Answers

1. Networking Fundamentals

1.1 The OSI Model

The OSI (Open Systems Interconnection) model is a 7-layer framework that standardizes how network communication works. Each layer has a specific function and communicates with the layers above and below it. Understanding this is critical because NetSentinel operates primarily at Layers 2-4.

Layer	Name	Function	Protocols	NetSentinel Relevance
7	Application	User-facing services	HTTP, FTP, DNS, SSH	Service identification
6	Presentation	Data formatting/encryption	SSL/TLS, JPEG	Minimal
5	Session	Connection management	NetBIOS, RPC	Minimal
4	Transport	End-to-end delivery	TCP, UDP	Port analysis, flags
3	Network	Routing & addressing	IP, ICMP, ARP	IP analysis, geolocation
2	Data Link	MAC addressing	Ethernet, WiFi	MAC address capture
1	Physical	Bits on wire	Cables, radio	Not applicable

1.2 TCP/IP Protocol Suite

While OSI is the theoretical model, TCP/IP is what the internet actually uses. It has 4 layers: Link (Ethernet frames), Internet (IP packets), Transport (TCP/UDP segments), and Application (HTTP, DNS, etc.). NetSentinel captures data at the Link layer and analyzes up through Application.

1.3 Key Protocols You Must Know

TCP (Transmission Control Protocol)

Connection-oriented, reliable delivery. Uses a 3-way handshake: SYN -> SYN-ACK -> ACK. Has flags: SYN (synchronize), ACK (acknowledge), FIN (finish), RST (reset), PSH (push), URG (urgent). NetSentinel analyzes these flags to detect scans. Example: A SYN-only packet without ACK response suggests a SYN scan.

UDP (User Datagram Protocol)

Connectionless, no handshake, faster but unreliable. Used for DNS (port 53), NTP (port 123), streaming, gaming, VoIP. No flags to analyze, but payload size and frequency can indicate attacks.

ICMP (Internet Control Message Protocol)

Used for diagnostics: ping (Type 8 = request, Type 0 = reply), traceroute (Type 11 = time exceeded). Large ICMP payloads can indicate data exfiltration (ICMP tunneling).

DNS (Domain Name System)

Translates domain names to IP addresses. Usually UDP port 53. Very long DNS queries (>60 characters) can indicate DNS tunneling, where attackers hide data inside DNS requests.

ARP (Address Resolution Protocol)

Maps IP addresses to MAC addresses on local networks. ARP spoofing is a common LAN attack.

HTTP/HTTPS

HTTP (port 80) is unencrypted web traffic. HTTPS (port 443) uses TLS/SSL encryption. NetSentinel can identify the service but cannot inspect HTTPS payload due to encryption.

1.4 TCP Flags — Critical for Threat Detection

TCP flags control how connections are established, maintained, and terminated. Attackers manipulate these flags for reconnaissance. NetSentinel detects these patterns:

Attack	Flags Pattern	What It Means
SYN Scan	SYN only (S)	Attacker sends SYN to check if port is open, never completes handshake
XMAS Scan	FIN+PSH+URG (FPU)	All flags lit up 'like a Christmas tree' — used to bypass firewalls
NULL Scan	No flags	Empty TCP packet — some systems respond differently to NULL, revealing info
FIN Scan	FIN only (F)	Sends FIN without established connection — stealthy port detection
SYN Flood	Many SYNs	Overwhelms server with connection requests — Denial of Service attack

2. Network Security Concepts

2.1 Intrusion Detection Systems (IDS)

An IDS monitors network traffic for suspicious activity. There are two main approaches:

Signature-Based IDS: Matches traffic against a database of known attack signatures. Fast and accurate for known threats but cannot detect zero-day (unknown) attacks. Example: Snort.

Anomaly-Based IDS: Learns 'normal' behavior and flags deviations. Can detect unknown attacks but has higher false positive rates. Our Isolation Forest model is anomaly-based.

NetSentinel is a hybrid: It uses rule-based detection (like signature-based) for known attack patterns (port scans, XMAS scans) AND ML-based anomaly detection (Isolation Forest) for unknown threats.

2.2 IDS vs IPS

IDS (Intrusion Detection System) only DETECTS and alerts. IPS (Intrusion Prevention System) can also BLOCK malicious traffic. NetSentinel is an IDS — it detects and explains but does not block traffic.

2.3 Types of Network Attacks

Reconnaissance/Scanning

Attacker probes the network to discover open ports, services, and vulnerabilities. Includes port scanning (SYN scan, XMAS scan), OS fingerprinting, and service enumeration.

Denial of Service (DoS/DDoS)

Overwhelms a target with traffic to make it unavailable. SYN flood sends thousands of SYN packets. DDoS uses multiple sources (botnets).

Exploits

Targets specific vulnerabilities in software or protocols. Can lead to remote code execution, privilege escalation, or data theft.

Backdoors

Hidden access points installed by attackers for persistent access. Often communicate on unusual ports (4444, 31337).

Fuzzers

Send malformed or random data to crash systems or find vulnerabilities.

DNS Tunneling

Hides data inside DNS queries/responses. Indicated by unusually long DNS queries.

2.4 Existing Tools Comparison

Feature	Wireshark	Snort	Zeek	NetSentinel
Packet Capture	Yes	Yes	Yes	Yes
Protocol Decode	Deep	Basic	Deep	Medium
Real-time	Yes	Yes	Yes	Yes
AI Classification	No	No	No	Yes (XGBoost)
Explainability	No	No	No	Yes (SHAP)

Plain English	No	No	No	Yes
IP Geolocation	Via plugin	No	No	Built-in
Zero-day Detection	No	No	Limited	Yes (Isolation Forest)
Ease of Use	Complex	Complex	Complex	Simple (web UI)
PDF Reports	Limited	No	No	Yes

3. The UNSW-NB15 Dataset

The UNSW-NB15 dataset was created by the IXIA PerfectStorm tool at the Cyber Range Lab of UNSW Canberra (University of New South Wales). It is one of the most widely used modern benchmarks for network intrusion detection research. It was created because older datasets like KDD99 and NSL-KDD are outdated and don't represent modern attack patterns.

3.1 Dataset Statistics

Property	Value
Total Records	2,540,044
Training Set	175,341
Testing Set	82,332
Total Features	49
Attack Types	9 categories + Normal
Source	IXIA PerfectStorm + tcpdump (100GB raw PCAP)
Feature Extraction	Argus, Bro-IDS + 12 custom algorithms
Year Created	2015 (still widely used in 2024-2025 research)

3.2 Attack Categories

Category	Description	Example
Normal	Legitimate network traffic	Web browsing, email
Fuzzers	Random/malformed data to crash systems	Buffer overflow attempts
Analysis	Port scan, spam, penetration attempts	Nmap scanning
Backdoors	Hidden access bypassing authentication	Trojan horse connections
DoS	Denial of Service attacks	SYN flood, Slowloris
Exploits	Known vulnerability exploitation	Heartbleed, Shellshock
Generic	Generic attack patterns	Hash collision attacks
Reconnaissance	Information gathering	OS fingerprinting
Shellcode	Code injected into running processes	Reverse shell payloads
Worms	Self-replicating malware	Network worm propagation

3.3 Key Features Explained

- dur:** Duration of the connection in seconds
- sbytes / dbytes:** Source-to-destination / destination-to-source bytes transferred
- sttl / dttl:** Source / destination Time-To-Live values
- sload / dload:** Source / destination bits per second
- spkts / dpkts:** Source / destination packet count
- rate:** Total packets per second in the connection
- sinpkt / dinpkt:** Source / destination inter-packet arrival time
- sjit / djit:** Source / destination jitter (variation in packet timing)

tcprtt: TCP round-trip time (setup time for connection)

ct_state_ttl: Number of connections with same state and TTL

ct_srv_src: Number of connections to same service from source

smean / dmean: Mean packet size from source / destination

is_sm_ips_ports: Whether source/destination have same IP and port

3.4 Why UNSW-NB15 Over Other Datasets?

KDD99 (1999): Too old, contains redundant records, doesn't represent modern attacks. **NSL-KDD:** Improved version of KDD99 but still outdated. **CIC-IDS2017:** Good alternative but UNSW-NB15 has more diverse attack types and is more widely cited. UNSW-NB15 provides a realistic mix of modern normal and attack traffic with 49 carefully engineered features.

4. Machine Learning Models

4.1 Random Forest

Random Forest is an ensemble learning method that constructs multiple decision trees during training and outputs the class that is the mode (majority vote) of the individual trees.

How it works: (1) Randomly sample N subsets of the training data with replacement (bootstrap). (2) For each subset, build a decision tree but at each split, only consider a random subset of features. (3) For prediction, each tree votes and the majority wins.

Why we use it: Robust against overfitting, handles high-dimensional data well, provides built-in feature importance, works well with imbalanced datasets, and requires minimal hyperparameter tuning.

Our hyperparameters: `n_estimators=100` (100 trees), `max_depth=20`, `min_samples_split=5`, `min_samples_leaf=2`. These balance accuracy with computational efficiency.

4.2 XGBoost (Extreme Gradient Boosting)

XGBoost builds trees sequentially. Each new tree focuses on correcting the errors made by the previous trees. It uses gradient descent to minimize a loss function.

How it works: (1) Start with an initial prediction. (2) Compute residuals (errors). (3) Fit a new tree to predict the residuals. (4) Add this tree to the ensemble with a learning rate. (5) Repeat. The learning rate (0.1) controls how much each tree contributes, preventing overfitting.

Why we use it: Consistently achieves top performance in tabular data competitions. Has built-in regularization (L1 and L2), handles missing values natively, and supports parallel computing.

Our hyperparameters: `n_estimators=100`, `max_depth=10`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8`. Subsample and colsample prevent overfitting by only using 80% of data/features per tree.

4.3 Isolation Forest (Anomaly Detection)

Isolation Forest is fundamentally different from RF and XGBoost. It's an UNSUPERVISED anomaly detection algorithm — it doesn't need labeled attack data to detect threats.

How it works: The key insight is that anomalies are 'few and different.' If you randomly partition data, anomalies will be isolated (separated) in fewer splits than normal points. (1) Randomly select a feature. (2) Randomly select a split value. (3) Repeat until each point is isolated. Points that are isolated quickly (short path length) are anomalies.

Why we use it: It can detect ZERO-DAY attacks — attacks the model has never seen before. Since it only learns what 'normal' looks like, anything significantly different is flagged. This is critical because new attack types emerge constantly.

Training approach: We train ONLY on normal traffic. During inference, the model scores how 'abnormal' each packet is. Score of -1 means anomaly (attack), +1 means normal.

4.4 Model Evaluation Metrics

Accuracy: $\text{Correct predictions} / \text{Total predictions}$. Can be misleading with imbalanced data.

Precision: $\text{True Positives} / (\text{True Positives} + \text{False Positives})$. 'Of all packets flagged as attacks, how many actually were?' High precision = few false alarms.

Recall (Sensitivity): $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$. 'Of all actual attacks, how many did we catch?' High recall = few missed attacks.

F1-Score: Harmonic mean of Precision and Recall: $2*(P*R)/(P+R)$. Balances both metrics. This is our PRIMARY metric.

AUC-ROC: Area Under the ROC Curve. Measures how well the model distinguishes between classes across all thresholds. 1.0 = perfect, 0.5 = random.

Confusion Matrix: 2x2 table showing True Positives, True Negatives, False Positives, False Negatives.

4.5 Feature Scaling — StandardScaler

StandardScaler transforms each feature to have mean=0 and standard deviation=1 using the formula: $z = (x - \text{mean}) / \text{std}$. This is necessary because features like 'sbytes' (can be millions) and 'is_ftp_login' (0 or 1) have vastly different scales. Without scaling, features with larger values would dominate the model's decisions. We fit the scaler on training data and apply it to test data.

5. Explainable AI (XAI)

XAI is one of the hottest research areas in ML right now. The core problem: ML models (especially deep learning) are 'black boxes' — they make accurate predictions but we can't understand WHY. In cybersecurity, this is dangerous: a security analyst needs to understand why an alert was triggered to know how to respond.

5.1 SHAP (SHapley Additive exPlanations)

SHAP is based on Shapley values from cooperative game theory (Nobel Prize winning concept from 1953). The idea: treat each feature as a 'player' in a 'game' of making predictions. The Shapley value tells you each player's fair contribution to the final prediction.

How it works: For each prediction, SHAP computes how much each feature pushed the prediction away from the average prediction. Positive SHAP value = feature pushed toward 'Attack'. Negative SHAP value = feature pushed toward 'Normal'.

Example: If SHAP says $ttl=0.45$ for a packet, it means the TTL value of this packet contributed $+0.45$ toward classifying it as an attack. This tells the analyst: 'This packet was flagged partly because its TTL value is unusual.'

Types of SHAP plots we generate:

- **Summary Plot (Beeswarm):** Shows every feature's impact across all samples. Each dot is one prediction. Red = high feature value, blue = low. Position on X-axis = impact on prediction.
- **Bar Plot:** Mean absolute SHAP value per feature — shows overall feature importance.
- **Waterfall Plot:** Explains one specific prediction step by step.
- **Force Plot:** Interactive visualization of how features push prediction from base value.

TreeExplainer: We use `shap.TreeExplainer` which is optimized for tree-based models (RF, XGBoost). It computes exact SHAP values efficiently using the tree structure, not approximations.

5.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME explains individual predictions by creating a simple, interpretable model (like linear regression) that approximates the complex model's behavior locally (near one specific data point). It works by: (1) Perturbing the input slightly to create variations. (2) Getting the complex model's predictions for all variations. (3) Fitting a simple linear model to these local predictions. (4) The linear model's coefficients explain the prediction.

SHAP vs LIME: SHAP provides theoretically grounded, consistent explanations (Shapley values satisfy uniqueness, symmetry, and additivity axioms). LIME is faster but can give different explanations for the same input depending on the perturbation. For our tool, SHAP is the primary explainer because of its mathematical rigor.

6. Tools & Libraries

Scapy

Python library for packet manipulation and sniffing. It can capture, decode, forge, and send network packets. We use it as our packet capture engine. It works at Layer 2, giving us access to Ethernet frames, IP headers, TCP/UDP ports, and payload data. Requires root/sudo privileges for live capture because it uses raw sockets.

Streamlit

Python framework for building interactive web applications quickly. We use it for our real-time dashboard. It automatically handles the frontend (HTML/CSS/JS) so we write only Python. Supports real-time updates, interactive widgets, charts, and data tables.

scikit-learn

The most widely used ML library in Python. We use it for: RandomForestClassifier, IsolationForest, StandardScaler, train_test_split, and all evaluation metrics. It provides a consistent API: fit(), predict(), predict_proba().

XGBoost

Optimized distributed gradient boosting library. Faster and more memory-efficient than scikit-learn's GradientBoosting. Created by Tianqi Chen at UW. Dominates ML competitions.

SHAP

Library by Scott Lundberg (Microsoft). Implements Shapley value-based explanations for ML models. TreeExplainer for tree models, KernelExplainer for any model.

Plotly

Interactive visualization library. We use it for the geographic map (scatter_geo), bar charts, pie charts, and other dashboard visualizations. Supports hover, zoom, and pan.

pandas

Data manipulation library. DataFrames are used throughout for handling packet data, dataset processing, and feature engineering.

NumPy

Numerical computing library. Used for array operations, feature scaling math, and SHAP value computations.

joblib

Used to save (serialize) and load trained ML models. Efficient for large numpy arrays inside sklearn models.

FPDF2 / ReportLab

PDF generation libraries. We use them to create the forensic analysis report with threat summaries, anomaly details, and protocol statistics.

ip-api.com

Free IP geolocation API. Given an IP address, returns country, city, ISP, and lat/lon coordinates. We use it to map where traffic is coming from/going to.

7. NetSentinel Architecture

7.1 System Pipeline

NetSentinel follows a 5-stage pipeline:

Stage 1 — CAPTURE: Scapy captures raw packets from the network interface (en0 on Mac). Each packet is a raw byte sequence containing Ethernet, IP, TCP/UDP, and payload layers.

Stage 2 — PARSE: PacketParser extracts structured information: source/destination IP and port, protocol, TCP flags, TTL, payload size, and service identification using well-known port numbers.

Stage 3 — ENRICH: GeoLocator queries ip-api.com to map IPs to geographic locations. FlowAggregator groups packets into bidirectional flows (sessions) to detect patterns across multiple packets.

Stage 4 — ANALYZE: Three-pronged analysis: (a) Rule-based engine checks for known suspicious patterns (SYN scans, XMAS scans, suspicious ports). (b) ML model (XGBoost) classifies as normal/attack. (c) Isolation Forest provides anomaly score.

Stage 5 — EXPLAIN & DISPLAY: AI Explainer generates human-readable explanations. SHAP shows why the ML model made its decision. Streamlit dashboard displays everything in real-time with color-coded threat levels.

7.2 Key Design Decisions

Why Streamlit over Flask?

Streamlit is faster to develop, has built-in real-time refresh, and handles all frontend complexity. For a 3-day project, it's the optimal choice.

Why XGBoost as primary model?

Consistently outperforms other models on tabular data. It's also compatible with SHAP's TreeExplainer for exact explanations.

Why rule-based + ML hybrid?

Rule-based catches known patterns instantly (zero latency). ML catches subtle, unknown patterns. Combining both gives comprehensive coverage.

Why Isolation Forest for anomaly detection?

It doesn't need labeled data, so it can detect attack types that weren't in the training set (zero-day detection).

Why flow aggregation?

Individual packets alone can't reveal attacks like port scans (which involve hundreds of packets). Grouping into flows reveals these patterns.

8. Research Context

8.1 Related Work

AI-based intrusion detection is one of the most active research areas in cybersecurity. Recent work (2024-2025) has focused on: deep learning for IDS (CNN, LSTM, GRU architectures), explainability (SHAP/LIME integration with IDS), federated learning for privacy-preserving IDS, and lightweight models for IoT/edge deployment.

Key papers in this space use UNSW-NB15 and CIC-IDS2017 as benchmarks. Most achieve >95% accuracy on UNSW-NB15 with Random Forest or XGBoost. The research gap we address: most papers focus purely on classification accuracy but don't build usable tools. NetSentinel bridges the gap between academic ML research and practical cybersecurity tooling.

8.2 Our Contributions

1. An integrated tool that combines live capture + ML classification + explainability in a single system
2. SHAP-based per-packet explanations that tell analysts WHY traffic was flagged
3. Hybrid detection: rule-based (known patterns) + supervised ML (XGBoost) + unsupervised anomaly detection (Isolation Forest)
4. Flow-level analysis for detecting multi-packet attacks (port scans, SYN floods)
5. Automated forensic PDF report generation
6. Designed for non-experts with plain English explanations and an intuitive web dashboard

9. Defense Q&A; — Potential Questions & Answers

Below are 50+ questions a professor might ask during your defense, with suggested answers.

TIP: Don't memorize answers word-for-word. Understand the concepts and explain in your own words.

Q: What is an Intrusion Detection System?

An IDS is a system that monitors network traffic for suspicious activity and generates alerts. It can be signature-based (matching known attack patterns) or anomaly-based (detecting deviations from normal behavior). NetSentinel is a hybrid IDS that uses both approaches.

Q: What's the difference between IDS and IPS?

IDS only detects and alerts. IPS can also take action to block malicious traffic. NetSentinel is an IDS — it identifies and explains threats but does not block traffic.

Q: Why did you choose the UNSW-NB15 dataset?

UNSW-NB15 is one of the most modern and comprehensive network intrusion datasets. Older alternatives like KDD99 (1999) don't represent modern attack patterns. UNSW-NB15 has 9 diverse attack types, 49 carefully engineered features, and is widely used in current research (2024-2025 papers still reference it). It's also freely available for academic use.

Q: What are the 9 attack types in UNSW-NB15?

Fuzzers (malformed data), Analysis (port scanning), Backdoors (hidden access), DoS (denial of service), Exploits (vulnerability exploitation), Generic (general attacks), Reconnaissance (information gathering), Shellcode (injected code), and Worms (self-replicating malware).

Q: How does Scapy capture packets?

Scapy uses raw sockets (which requires root/sudo privileges) to capture packets at the data link layer (Layer 2). It calls the operating system's packet capture API (libpcap on Mac/Linux) through its sniff() function. Each captured packet is a layered object — we can access Ethernet, IP, TCP/UDP, and payload layers programmatically.

Q: Why does packet capture need sudo/root?

Normal user processes can only open network sockets for sending/receiving data they're involved in. Capturing ALL packets on an interface (promiscuous mode) requires raw socket access, which is a privileged operation restricted to root for security reasons.

Q: Explain the TCP 3-way handshake.

Step 1: Client sends SYN (synchronize) to server. Step 2: Server responds with SYN-ACK. Step 3: Client sends ACK. Connection is now established. NetSentinel detects when only SYN is sent without completing the handshake — this indicates a SYN scan.

Q: What is a SYN scan and how do you detect it?

A SYN scan (half-open scan) sends SYN packets to ports without completing the handshake. If the port responds with SYN-ACK, it's open. The attacker then sends RST instead of ACK. We detect it by looking for TCP packets with only the SYN flag set, especially when many are sent to different ports from the same source.

Q: What is an XMAS scan?

An XMAS scan sets the FIN, PSF, and URG flags simultaneously — lighting up 'like a Christmas tree.' Some systems respond differently to these illegal flag combinations, revealing information about the OS or open ports. We detect it by checking if all three flags are present.

Q: Why did you choose Random Forest, XGBoost, and Isolation Forest?

Random Forest: robust, interpretable, provides feature importance. XGBoost: highest accuracy on tabular data, compatible with SHAP for explainability. Isolation Forest: unsupervised anomaly detection for zero-day attacks. Together, they cover supervised classification AND unsupervised anomaly detection.

Q: What is the difference between Random Forest and XGBoost?

Random Forest trains trees independently in parallel (bagging). Each tree sees a random subset of data and features. Final prediction is majority vote. XGBoost trains trees sequentially (boosting). Each new tree corrects errors of the previous ones. XGBoost generally achieves higher accuracy but is more prone to overfitting without tuning.

Q: How does Isolation Forest detect anomalies?

It randomly selects features and split values to partition data. Anomalies, being few and different, are isolated in fewer splits. Normal points need many splits. The anomaly score is based on the average path length. We train it ONLY on normal traffic, so any traffic that's significantly different from normal is flagged as potentially malicious.

Q: Can your system detect zero-day attacks?

Yes, through the Isolation Forest. Since it learns what 'normal' looks like (not what attacks look like), any traffic that deviates significantly from normal patterns will be flagged, even if it's a completely new attack type never seen in training data.

Q: What is SHAP and why is it important?

SHAP is based on Shapley values from game theory. It tells us exactly how much each feature contributed to a specific prediction. In cybersecurity, this is critical because analysts need to understand WHY an alert was triggered to know how to respond. A black-box 'this is an attack' is not enough — they need 'this is an attack because the TTL is unusually low and the packet rate is abnormally high.'

Q: What's the difference between SHAP and LIME?

SHAP: Based on Shapley values, provides theoretically consistent explanations, works globally and locally. Computationally expensive for non-tree models. LIME: Creates local linear approximations by perturbing input. Faster but can give inconsistent explanations. We primarily use SHAP (TreeExplainer) because it's exact for tree models.

Q: What is StandardScaler and why is it needed?

StandardScaler normalizes features to have mean=0 and std=1. Features like 'sbytes' can be millions while 'is_ftp_login' is 0 or 1. Without scaling, large-valued features would dominate. It's especially important for distance-based algorithms and neural networks.

Q: How did you handle missing values?

We filled missing values with 0 and replaced infinity values with 0. We chose 0-imputation because in network traffic context, missing values typically mean 'not applicable' or 'not observed' for that particular connection. For example, TCP-specific features would be 0 for UDP connections.

Q: What is your train/test split ratio?

80% training, 20% testing, with stratified sampling to maintain the same attack ratio in both sets. This ensures the model is evaluated on a representative sample.

Q: Why use F1-score as the primary metric instead of accuracy?

The dataset is imbalanced (more normal traffic than attack traffic). With imbalanced data, a model that predicts everything as 'normal' would have high accuracy but zero recall for attacks. F1-score balances precision (avoiding false alarms) and recall (catching actual attacks).

Q: What is AUC-ROC?

AUC-ROC measures how well the model distinguishes between classes across all possible classification thresholds. An AUC of 1.0 means perfect separation, 0.5 means random guessing. It's

threshold-independent, making it a robust metric for comparing models.

Q: How does flow aggregation help detect attacks?

Individual packets often look innocent. But patterns across multiple packets reveal attacks. A port scan involves hundreds of SYN packets to different ports from the same source. A SYN flood involves thousands of SYN packets to the same destination. By grouping packets into flows (bidirectional sessions), we can count unique ports contacted, packet rates, and flag distribution — revealing these multi-packet attacks.

Q: What is DNS tunneling and how do you detect it?

DNS tunneling hides data inside DNS queries. Normal DNS queries are short (e.g., 'google.com'). Tunneling queries are very long (60+ characters) because they encode data in the subdomain. We detect it by checking the length of DNS queries and flagging unusually long ones.

Q: Why did you use Streamlit instead of Flask/Django?

Development speed. Streamlit creates interactive web dashboards in pure Python with minimal code. For a 3-day deadline, it's the optimal choice. Flask would require separate HTML/CSS/JS templates. Streamlit handles real-time data updates, widgets, and charts out of the box.

Q: How does the geolocation feature work?

We use the free ip-api.com service. For each external IP address (not private/local IPs), we send an HTTP request to their API, which returns country, city, ISP, and lat/lon coordinates. Results are cached to avoid redundant API calls for the same IP.

Q: What are the limitations of your system?

1. Cannot inspect encrypted (HTTPS) traffic payload. 2. IP geolocation depends on external API. 3. ML model is trained on UNSW-NB15 which may not perfectly represent all real-world traffic. 4. Live capture requires root privileges. 5. Single-machine deployment, not distributed. 6. Feature mapping from live packets to dataset features is approximate.

Q: What is the novelty of your project compared to existing tools?

Existing tools like Wireshark show raw packet data but don't explain it. Existing ML-based IDS papers classify traffic but don't build usable tools. NetSentinel uniquely combines: live capture + protocol decoding + ML classification + SHAP explainability + geolocation + interactive dashboard + forensic reports — all in one tool.

Q: How would you improve this system in the future?

1. Deep learning models (LSTM for sequential packet analysis). 2. Deep packet inspection for unencrypted traffic. 3. Real-time model retraining with new traffic. 4. Integration with threat intelligence feeds. 5. Distributed deployment for enterprise networks. 6. IPS capabilities (blocking malicious traffic). 7. Training on CIC-IDS2017 for cross-dataset validation.

Q: What is the confusion matrix telling us?

The confusion matrix has 4 quadrants: True Positives (correctly caught attacks), True Negatives (correctly identified normal traffic), False Positives (normal traffic wrongly flagged as attack — false alarms), and False Negatives (attacks that slipped through undetected). In security, False Negatives are most dangerous — a missed attack can be catastrophic.

Q: Why did you use 100 trees in Random Forest?

100 trees provide a good balance between accuracy and training time. Beyond ~100 trees, the marginal improvement in accuracy is very small while training time increases linearly. This is a well-established default in the ML community.

Q: What is the contamination parameter in Isolation Forest?

Contamination tells the model what percentage of the training data is expected to be anomalous. We set it to the actual attack ratio in our training set. This helps the model calibrate its anomaly threshold. Setting it too low would miss attacks, too high would generate false alarms.

Q: Can your tool be used in a real enterprise environment?

As a proof of concept, yes. For production deployment, you'd need: higher performance (C/C++ capture instead of Python Scapy), distributed architecture, integration with SIEM systems, and continuous model retraining. But the core concepts — hybrid detection, explainability, and user-friendly dashboard — are directly applicable.

Good Luck with Your Defense!

Remember: Professors respect honesty. If you don't know something, say 'That's a great question, I would need to research that further' rather than guessing. Show enthusiasm for the project and explain the concepts in your own words. The fact that you built a working tool already puts you ahead of most submissions.